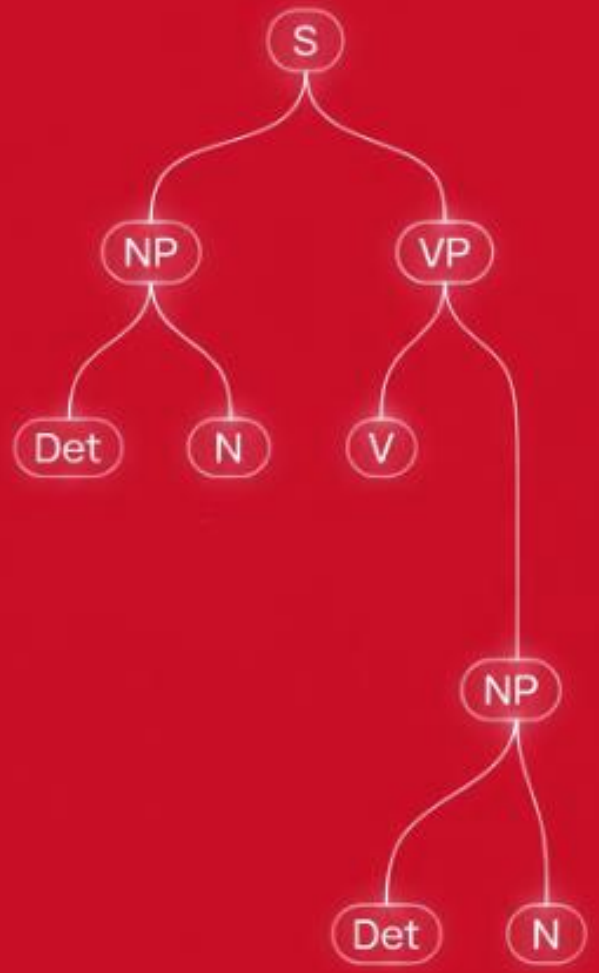
Cambridge Summer School

Introduction to Natural Language Processing

Dr Mariano Felice

7th August 2026



Outline

What is Natural Language Processing?
Basic text processing
- Tokenisation
- Part-of-Speech tagging
- Parsing

Representing words
- Bag of words
- Embeddings
- Building ML models

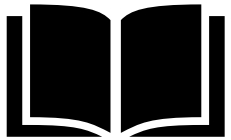
Language models
Group assignments

Humans and language

Humans

speak

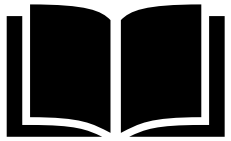
listen



read

write

Humans and language

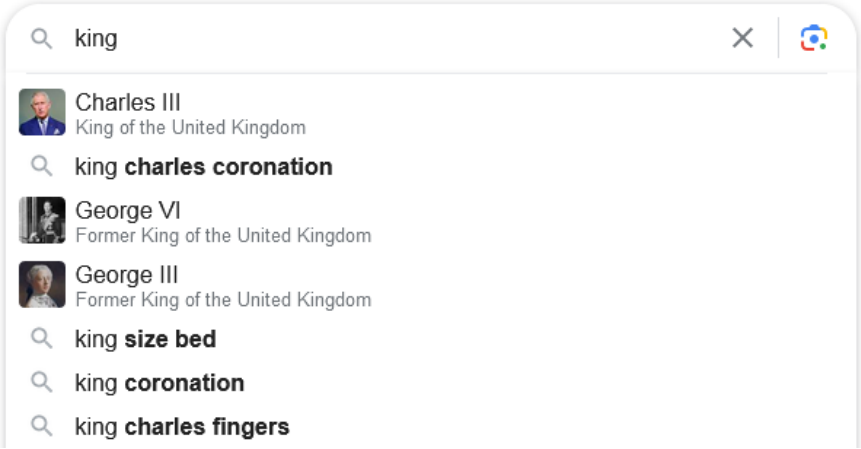
Humans				
	speak	**listen**	**read**	**write**
Machines	text-to-speech	speech recognition	text analysis e.g. parsing, document classification	text generation e.g. summarisation, machine translation

What is Natural Language Processing (NLP)?

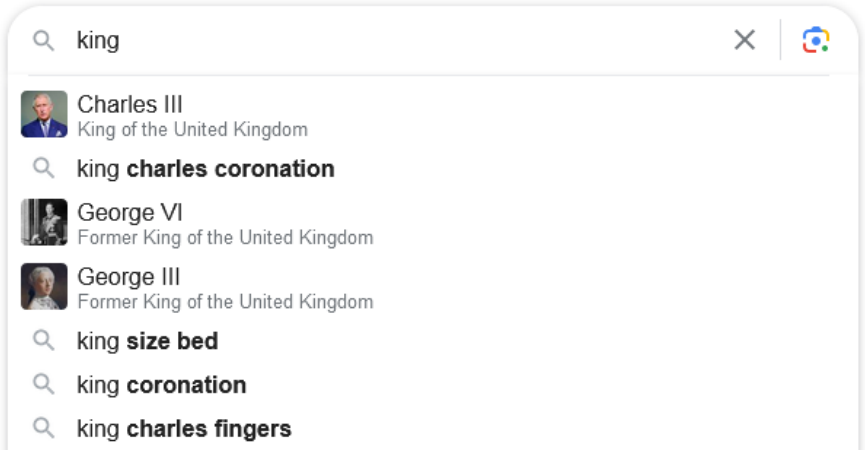
NLP is the sub-field of AI that deals with interpreting, understanding and producing human language (text and speech).

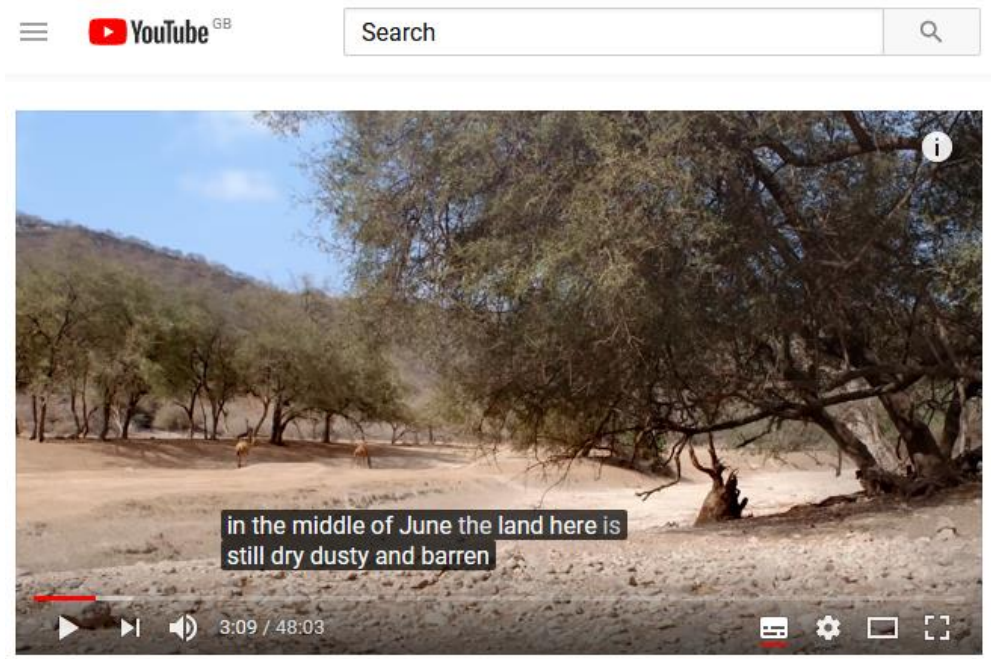
NLP is everywhere

NLP is everywhere

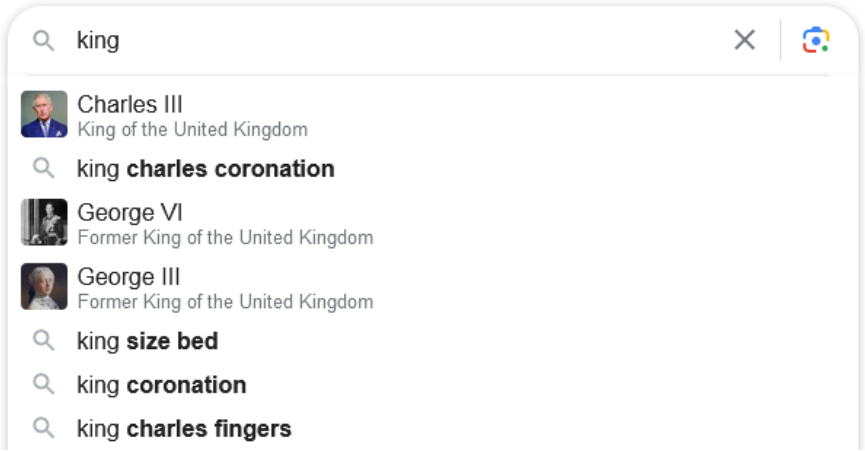


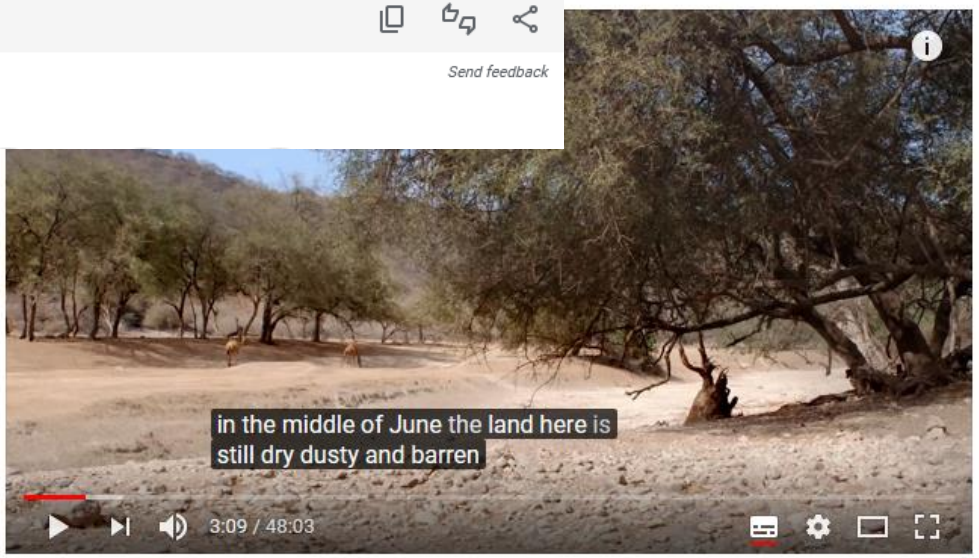
NLP is everywhere



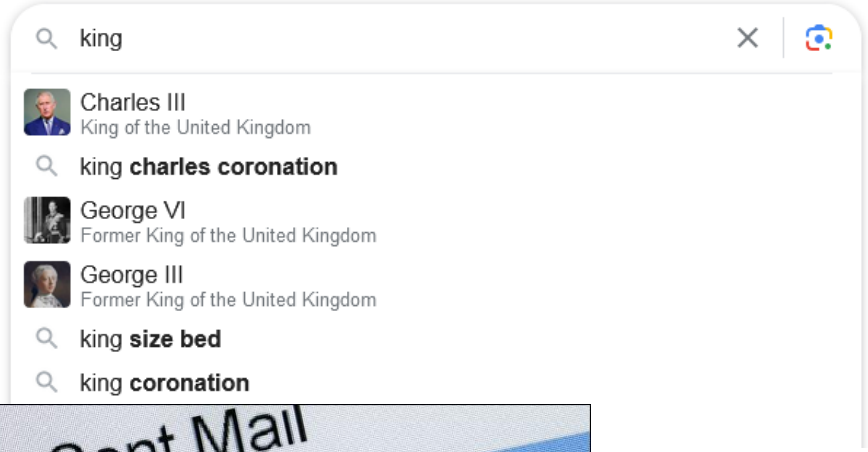


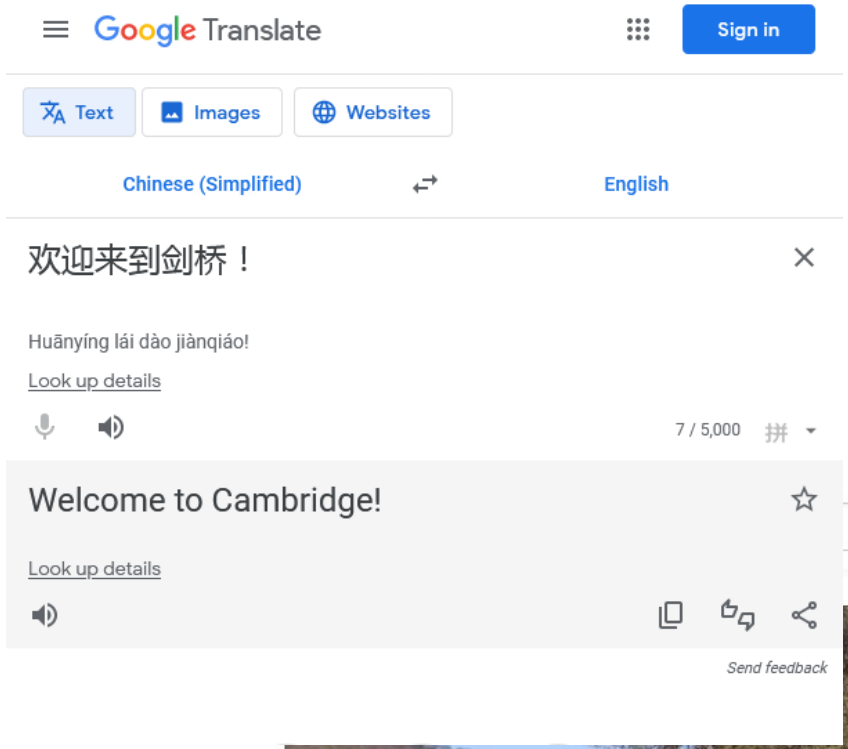
NLP is everywhere

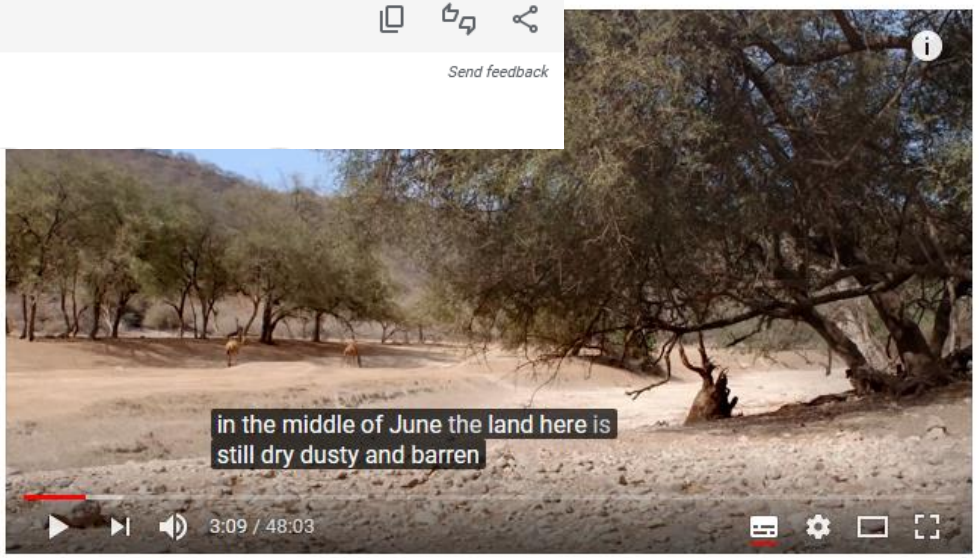




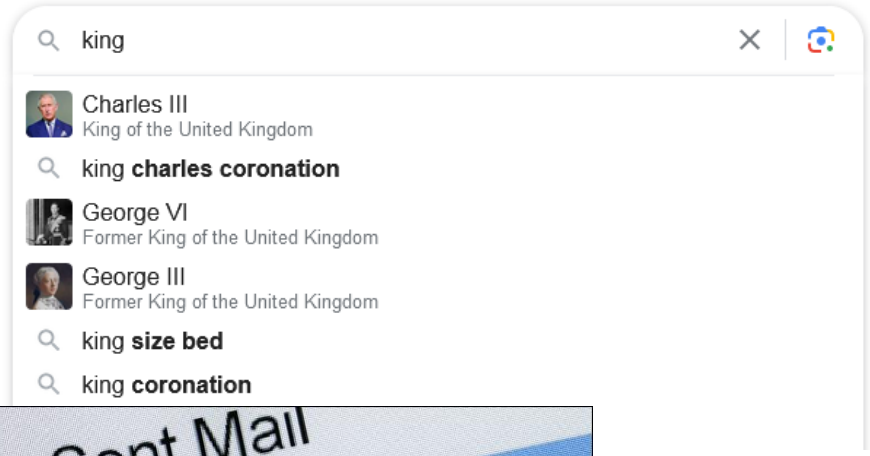
NLP is everywhere

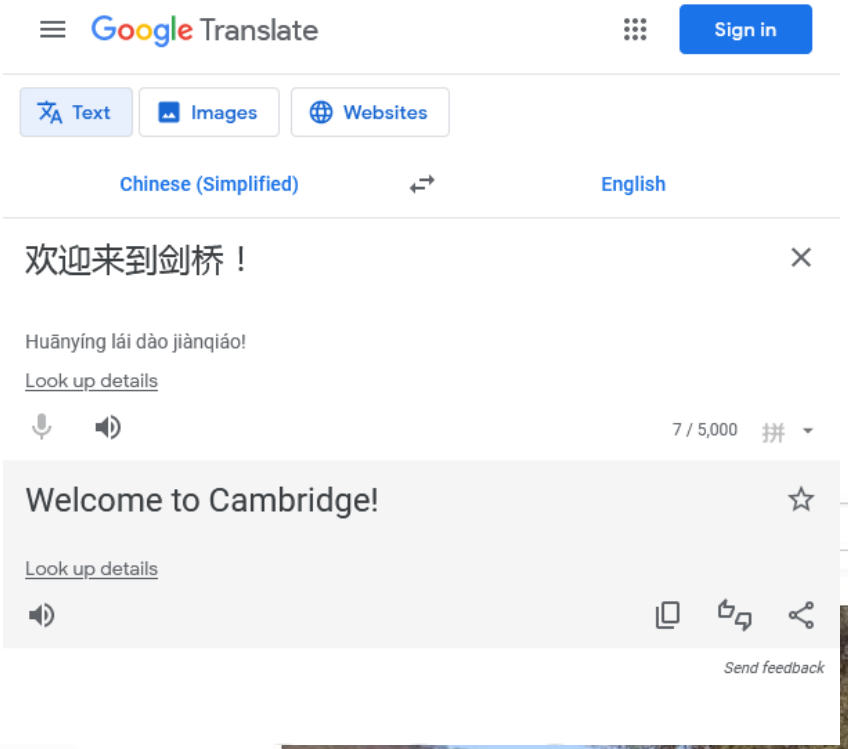


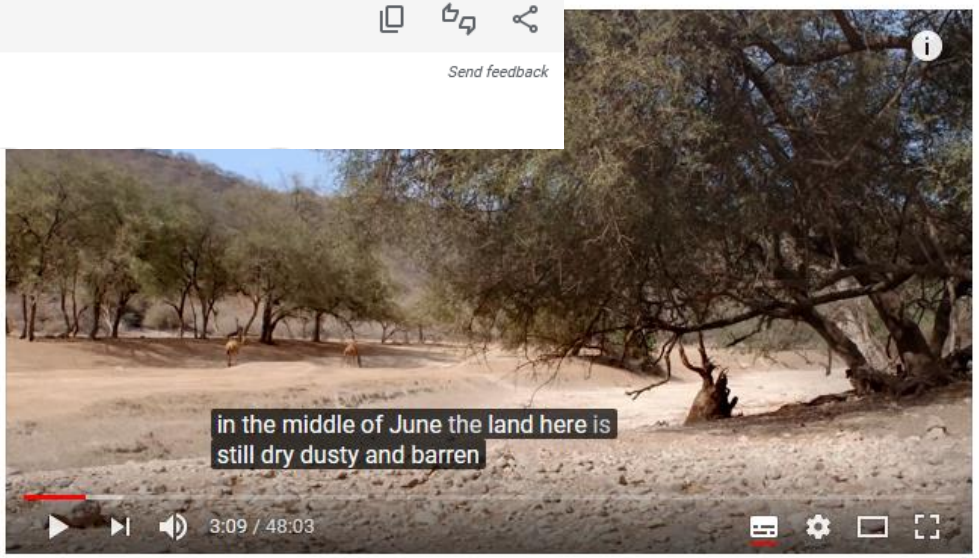




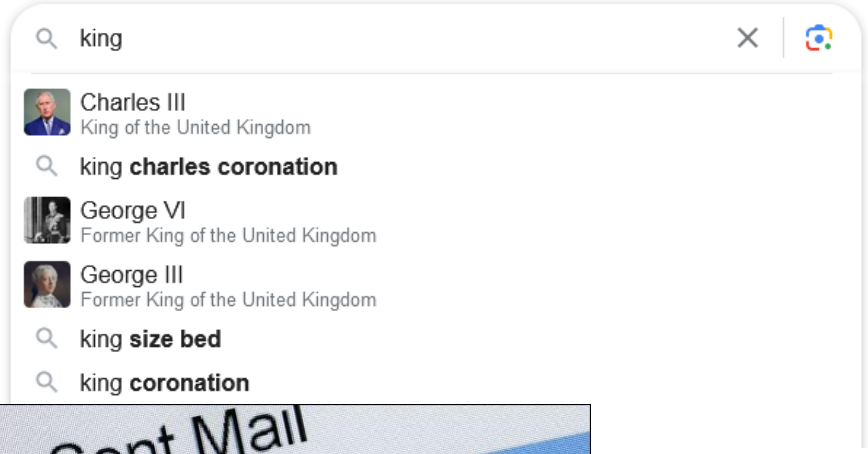
NLP is everywhere

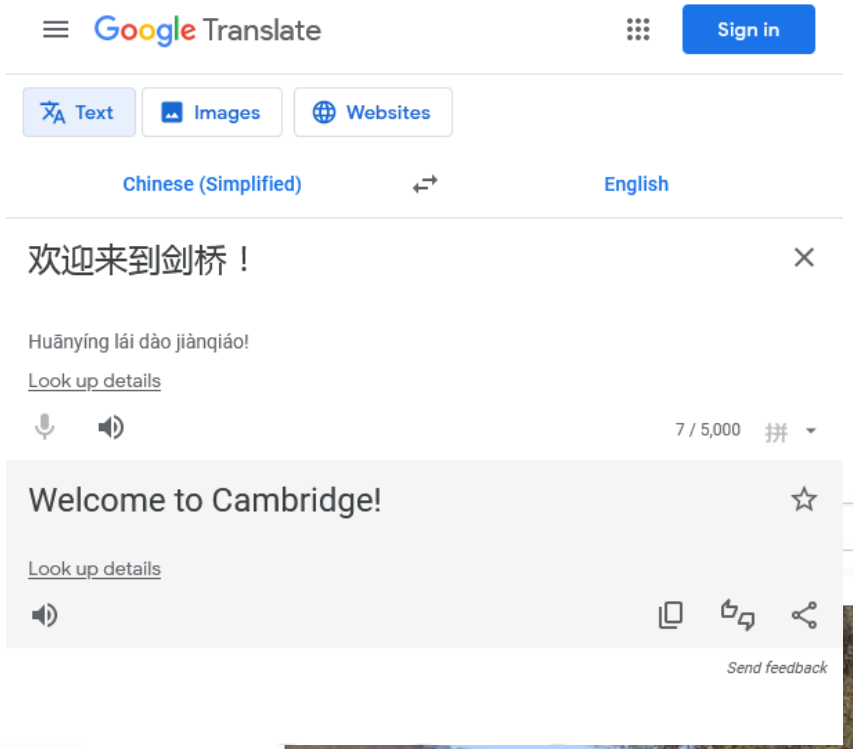




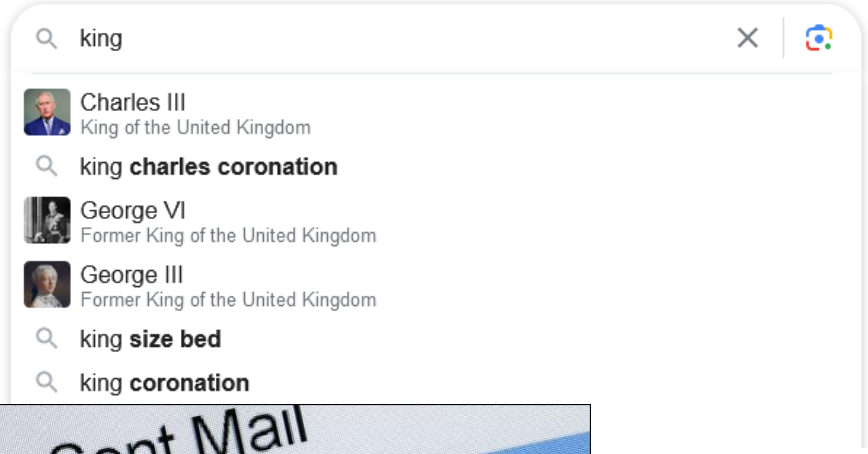


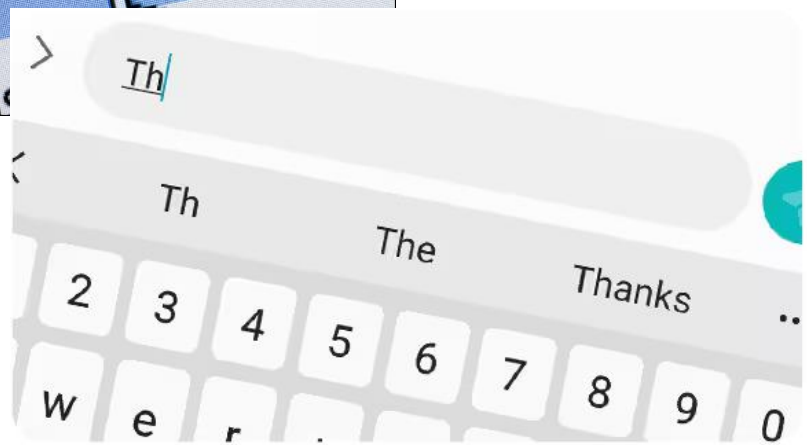
NLP is everywhere

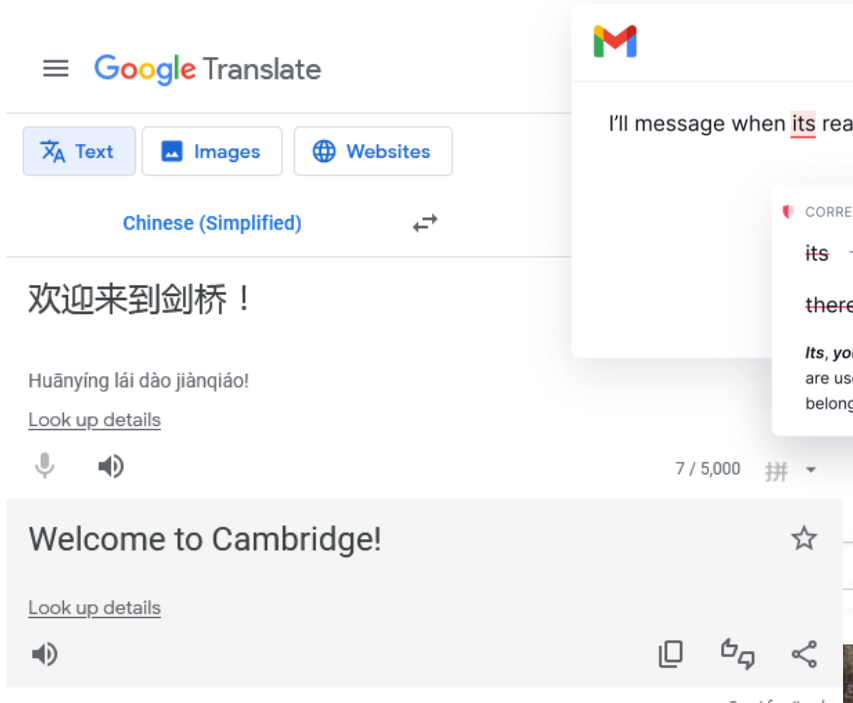


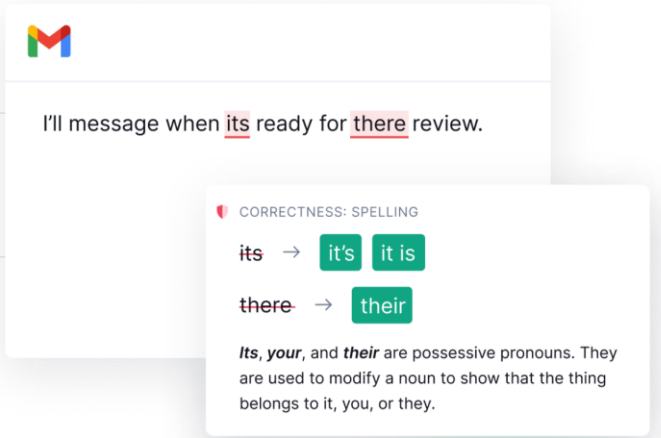


NLP is everywhere

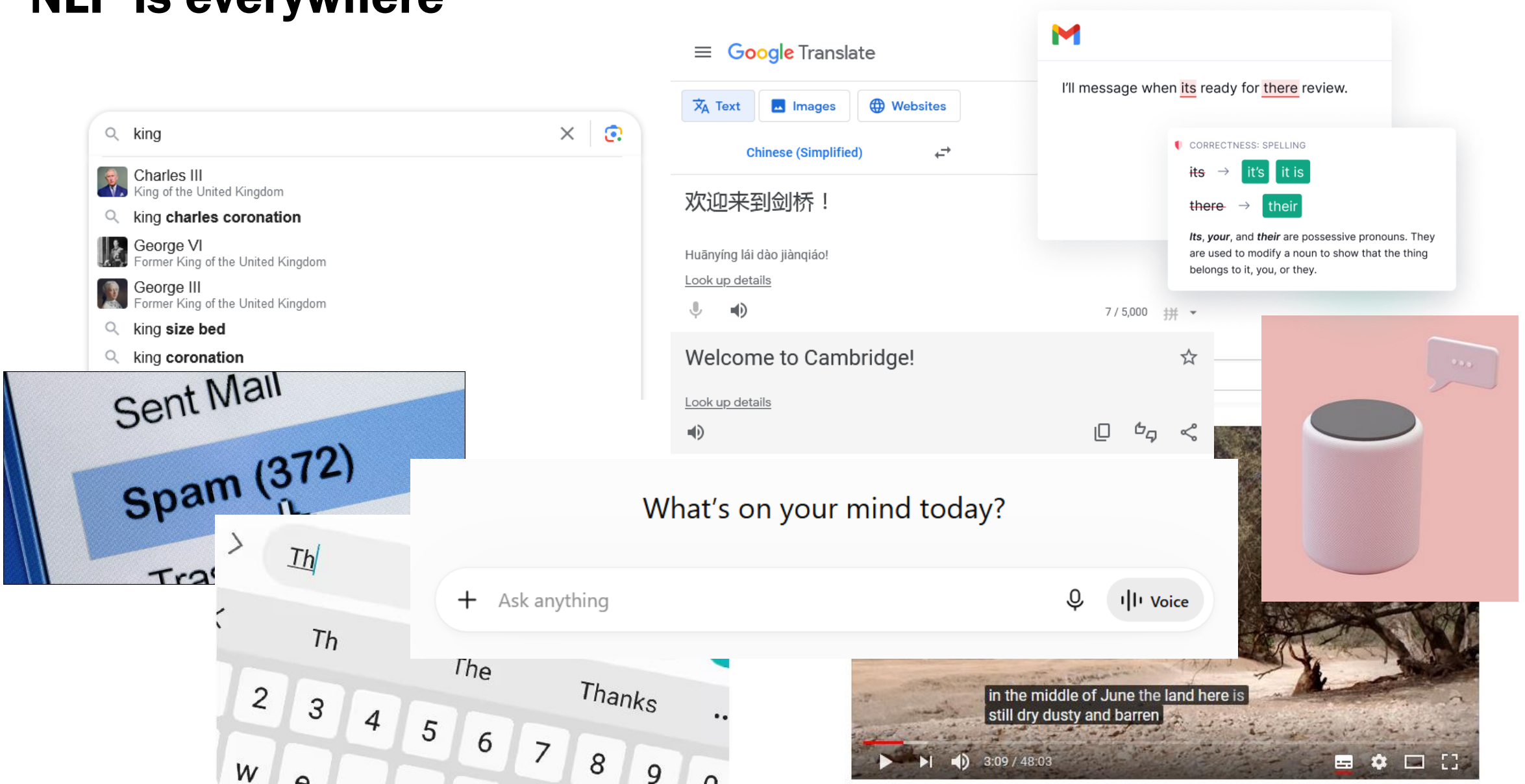




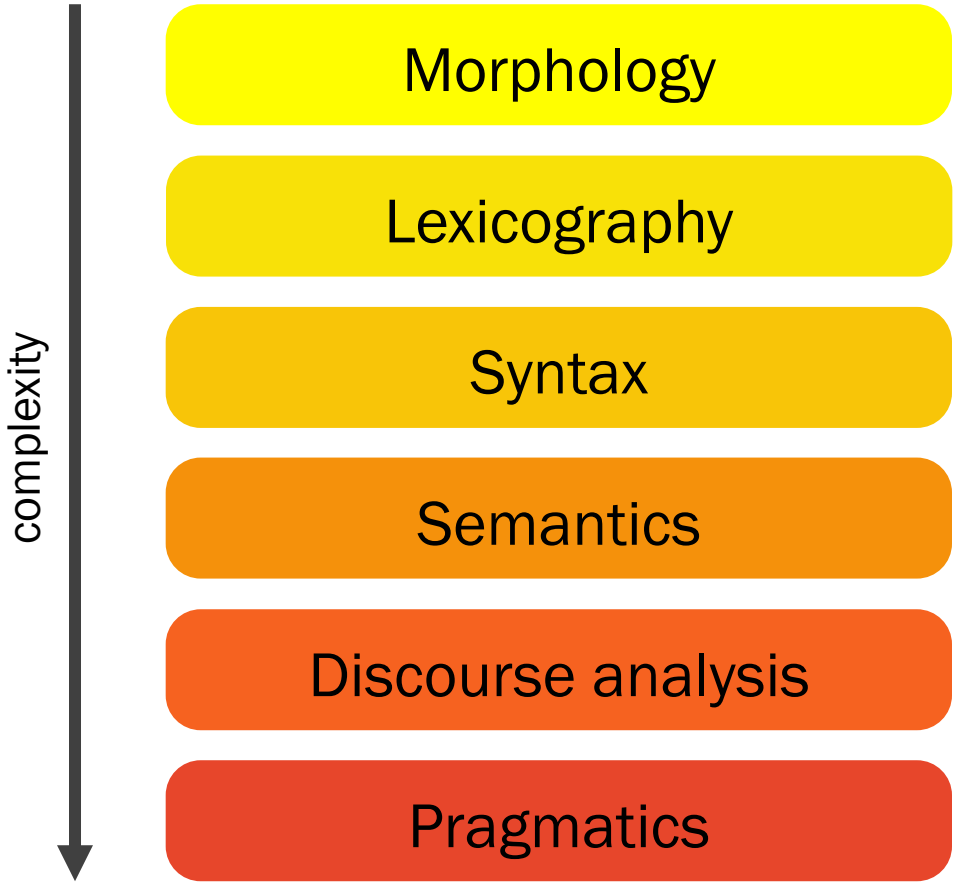




NLP is everywhere



Text processing



Basic text processing

Raw text

In the last three months, Facebook’s usually steep user growth didn’t change at all in the U.S. and fell in Europe following new data privacy laws. But during a second-quarter earnings report on Wednesday, July 25, the social media giant shared a new number — there are 2.5 billion people using a Facebook-owned app, which also includes Instagram, WhatsApp, and Messenger, each month.

Tokenisation

In the last three months , Facebook ’s usually steep user growth did n’t change at all in the U.S. and fell in Europe following new data privacy laws . But during a second-quarter earnings report on Wednesday , July 25 , the social media giant shared a new number — there are 2.5 billion people using a Facebook-owned app , which also includes Instagram , WhatsApp , and Messenger , each month .

Tokenisation

- Unclear cases:

Facebook’s → Facebook | ’s

didn’t → did | n’t

did | not

second-quarter → second | - | quarter

- Language-specific

- Logographies (e.g. Chinese) ⇨ MaxMatch

他特别喜欢北京烤鸭

他	特别	喜欢	北京烤鸭
He	especially	likes	Pekin duck

Sentence segmentation

In the last three months , Facebook ’s usually steep user growth did n’t change at all in the U.S. and fell in Europe following new data privacy laws . But during a second-quarter earnings report on Wednesday , July 25 , the social media giant shared a new number — there are 2.5 billion people using a Facebook-owned app , which also includes Instagram , WhatsApp , and Messenger , each month .

Part-of-speech tagging

In the last three months , Facebook ’s usually steep user growth did n’t change at all in the U.S. and fell in Europe following new data privacy laws .

Part-of-speech tagging

In the last three months , Facebook ’s usually steep user growth

PREP

did n’t change at all in the U.S. and fell in Europe following new

data privacy laws .

Part-of-speech tagging

In the last three months , Facebook ’s usually steep user growth

PREP DET

did n’t change at all in the U.S. and fell in Europe following new

data privacy laws .

Part-of-speech tagging

In the last three months , Facebook ’s usually steep user growth

PREP DET ADJ

did n’t change at all in the U.S. and fell in Europe following new

data privacy laws .

Part-of-speech tagging

Shallow parsing/chunking

In the last three months , Facebook ’s usually steep user growth did n’t change at all in the U.S. and fell in Europe following new data privacy laws .

Shallow parsing/chunking

In the last three months , Facebook ’s usually steep user growth

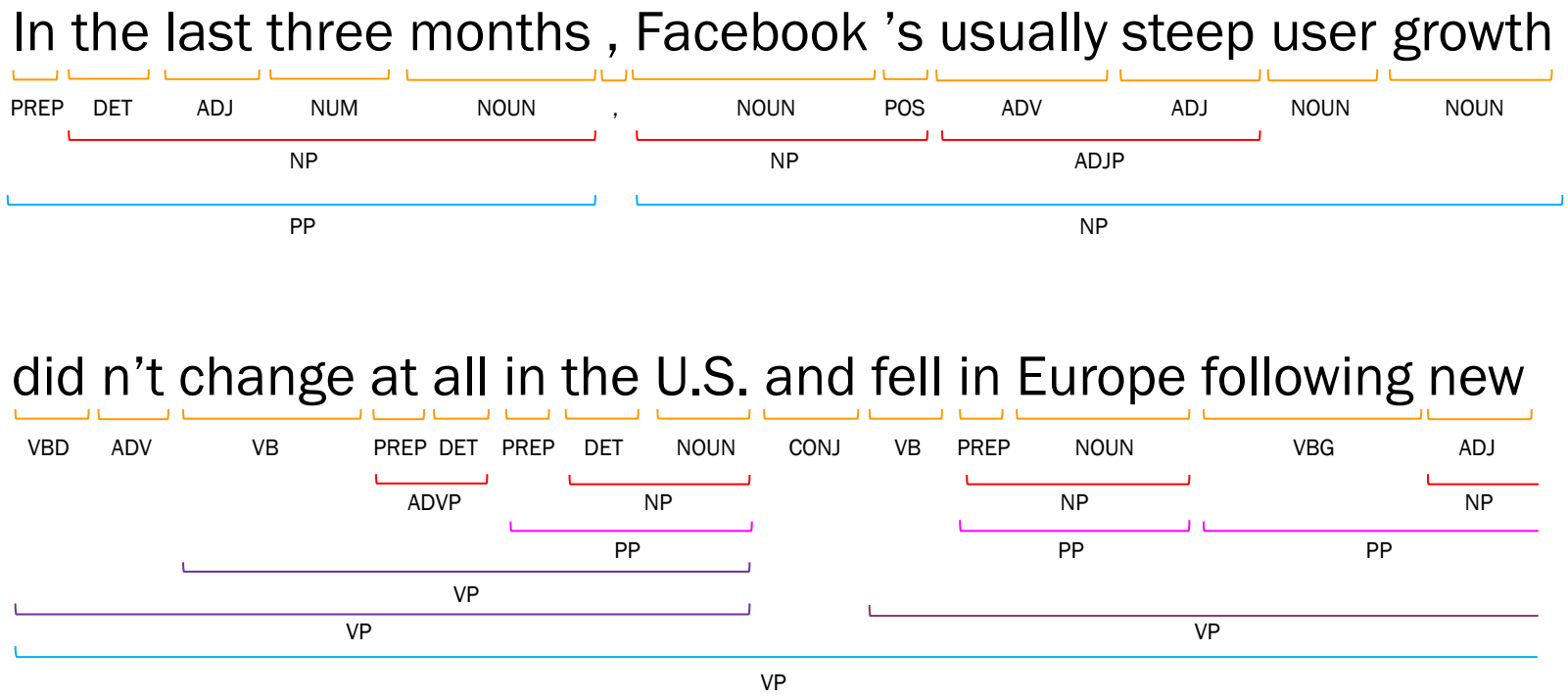
PP NP

did n’t change at all in the U.S. and fell in Europe following new

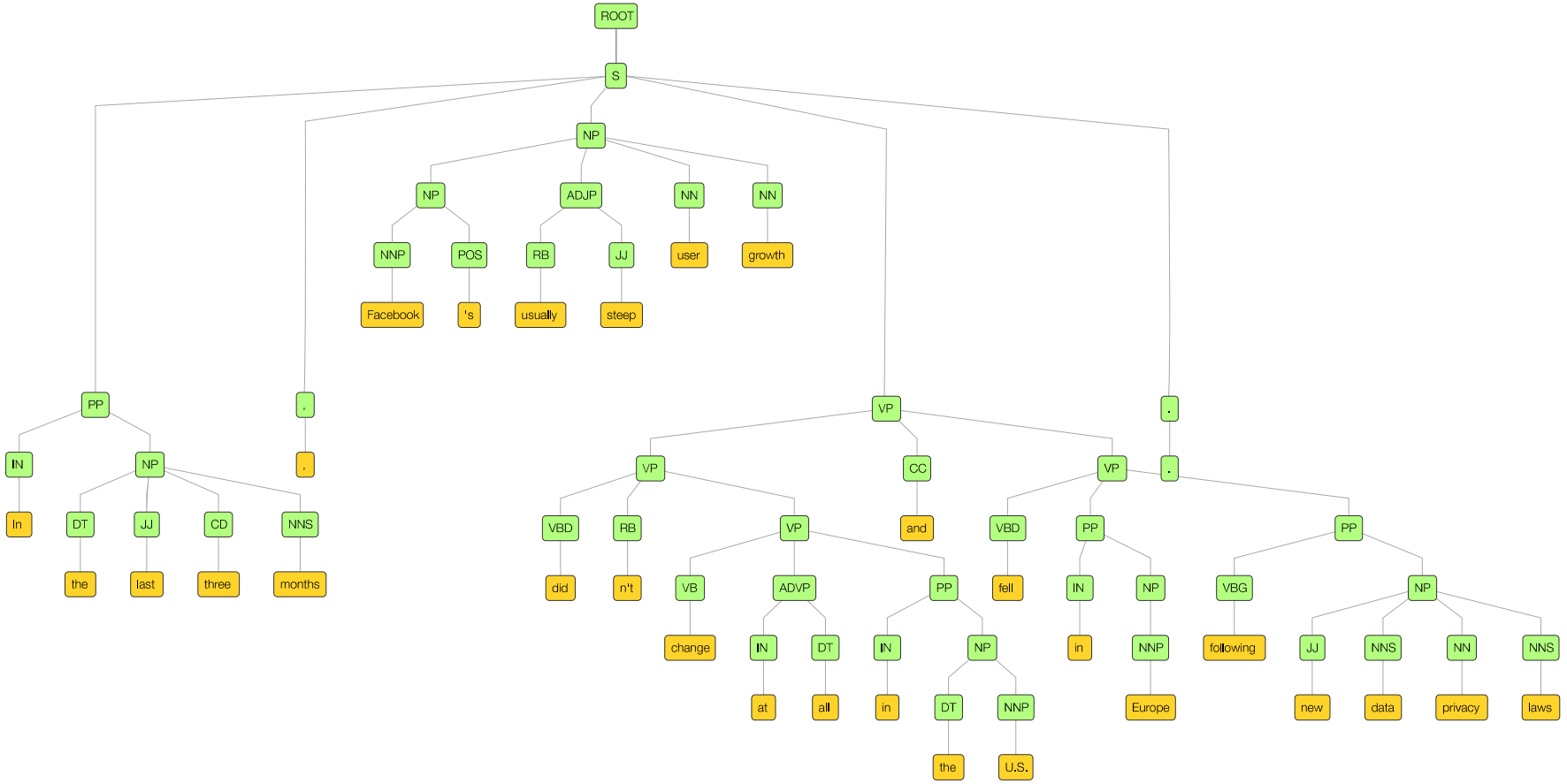
VP

data privacy laws .

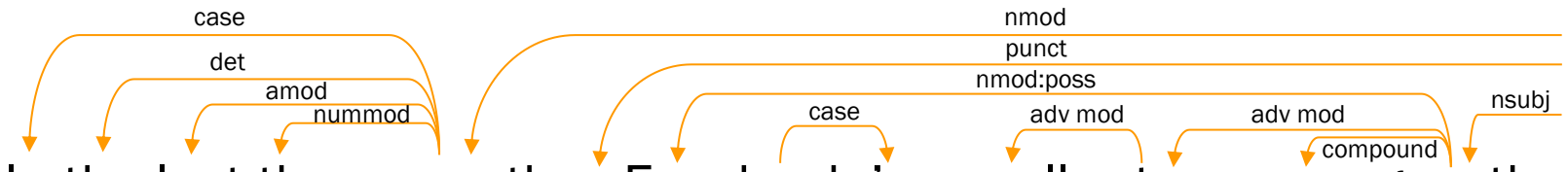
Constituency parsing

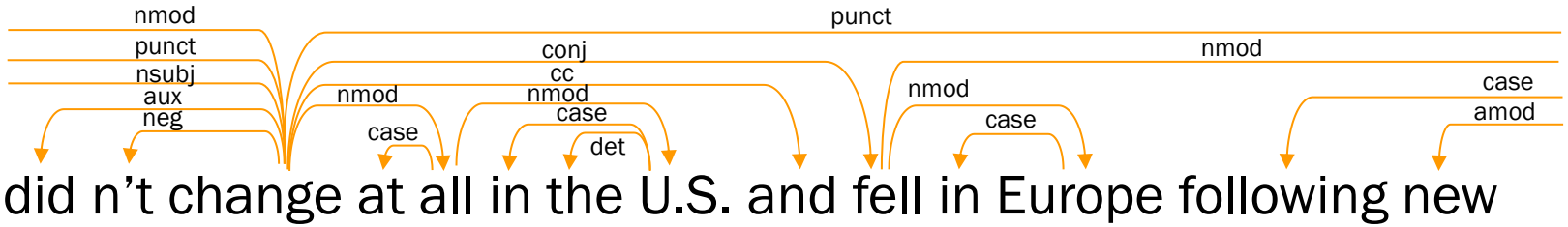


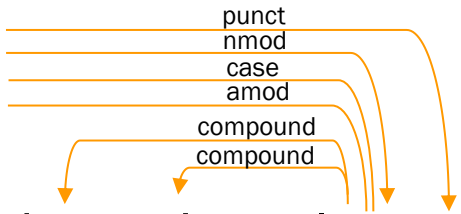
Constituency parsing



Dependency parsing







Let’s try it!

s.id/css26

Stemming and lemmatisation

Stemming

- Reduce a word to its stem (≈ morphological root).
- Remove common endings, such as -s, -es, -ing, -ity, -ly, -ation, etc.
- The stem might not be a valid word.

Lemmatisation

- Reduce a word to its lemma (or dictionary form).
- The lemma is always a valid word.

Stemming and lemmatisation

Word	Stem (Porter)	Lemma
books		

Stemming and lemmatisation

Word	Stem (Porter)	Lemma
books	book	book

Stemming and lemmatisation

Word	Stem (Porter)	Lemma
books	book	book
walking		

Stemming and lemmatisation

Word	Stem (Porter)	Lemma
books	book	book
walking	walk	walk

Stemming and lemmatisation

Word	Stem (Porter)	Lemma
books	book	book
walking	walk	walk
broke		

Stemming and lemmatisation

Word	Stem (Porter)	Lemma
books	book	book
walking	walk	walk
broke	broke	break

Stemming and lemmatisation

Word	Stem (Porter)	Lemma
books	book	book
walking	walk	walk
broke	broke	break
university		

Stemming and lemmatisation

Word	Stem (Porter)	Lemma
books	book	book
walking	walk	walk
broke	broke	break
university	univers	university

Stemming and lemmatisation

Word	Stem (Porter)	Lemma
books	book	book
walking	walk	walk
broke	broke	break
university	univers	university
ponies		

Stemming and lemmatisation

Word	Stem (Porter)	Lemma
books	book	book
walking	walk	walk
broke	broke	break
university	univers	university
ponies	poni	pony

Stemming and lemmatisation

Word	Stem (Porter)	Lemma
books	book	book
walking	walk	walk
broke	broke	break
university	univers	university
ponies	poni	pony
has		

Stemming and lemmatisation

Word	Stem (Porter)	Lemma
books	book	book
walking	walk	walk
broke	broke	break
university	univers	university
ponies	poni	pony
has	ha	have

Stemming and lemmatisation

Word	Stem (Porter)	Lemma
books	book	book
walking	walk	walk
broke	broke	break
university	univers	university
ponies	poni	pony
has	ha	have
better		

Stemming and lemmatisation

Word	Stem (Porter)	Lemma
books	book	book
walking	walk	walk
broke	broke	break
university	univers	university
ponies	poni	pony
has	ha	have
better	better	good

Let’s try it!

s.id/css26

Representing words and text

How do we represent words for machine learning?

- Remember, machines don’t understand text; they only understand numbers.
- We usually represent words and documents as vectors, since they provide rich information and are great for matrix operations in machine learning.
- General approach:

Document representation = Word representations + Aggregation

Words #1: One-Hot Vectors

Assign every word a unique slot in our vocabulary of size *V*.


`cat    = [1, 0, 0, 0, …, 0]`


`kitten = [0, 0, 0, 1, …, 0]`


`dog    = [0, 1, 0, 0, …, 0]`


`house  = [0, 0, 1, 0, …, 0]`

- Vectors are **sparse**: they are mostly zeros with a single 1.
- The position of the value 1 indicates which word it is.
- Vectors are **high-dimensional**, i.e. their length equals our vocabulary size (e.g. 50,000 words).
- Vectors don’t share any information, so the relationship between **cat** and **kitten** is the same as **cat** and **house** (no notion of similarity).

Document #1: Bag of Words (BoW)

Sum or count the One-Hot vectors for all words in the document.

```
cat     = [1, 0, 0, 0, …, 0]
house   = [0, 0, 1, 0, …, 0]
cat     = [1, 0, 0, 0, …, 0]
-----------------------------
V_doc   = [2, 0, 1, 0, …, 0]
```

- If words are missing from our vocabulary, their vectors will be all zeros, e.g. `V = [0, 0, 0, 0, …, 0]`
- Ignores word order, so `V("the dog chased the cat") = V("the cat chased the dog")`.

Words #2: Word embeddings (Word2Vec)

Dense vectors of *n* dimensions (e.g. *n* = 300).

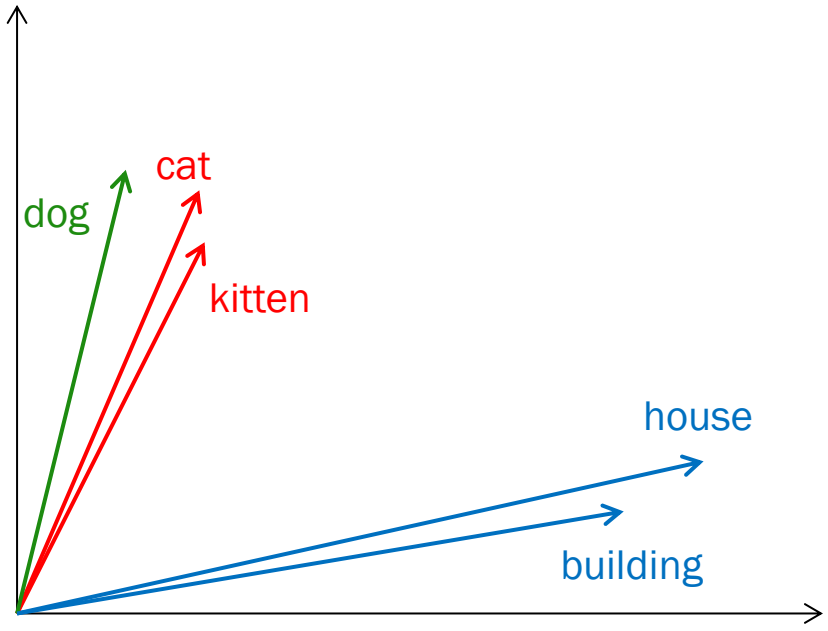
```
cat    = [ 0.92,  0.99, 0.59, -0.91, …, 0.32]
kitten = [ 0.91,  0.99, 0.08, -0.90, …, 0.11]
dog    = [ 0.92,  0.21, 0.62, -0.92, …, 0.44]
house  = [-0.45, -0.72, 0.66,  0.98, …, 0.84]
```

- These numbers are calculated by looking at surrounding words. Words that appear in similar contexts will have similar values.
- Each dimension represents an abstract attribute, e.g.

```
[animal, feline, age, place, …, size]
```

Words #2: Word embeddings (Word2Vec)

- If we plot these vectors in a multidimensional space, we'll see that similar words are closer together.



- Vectors capture relationships between words:

```
kitten – cat + dog = puppy
king – man + woman = ? ? ?
```

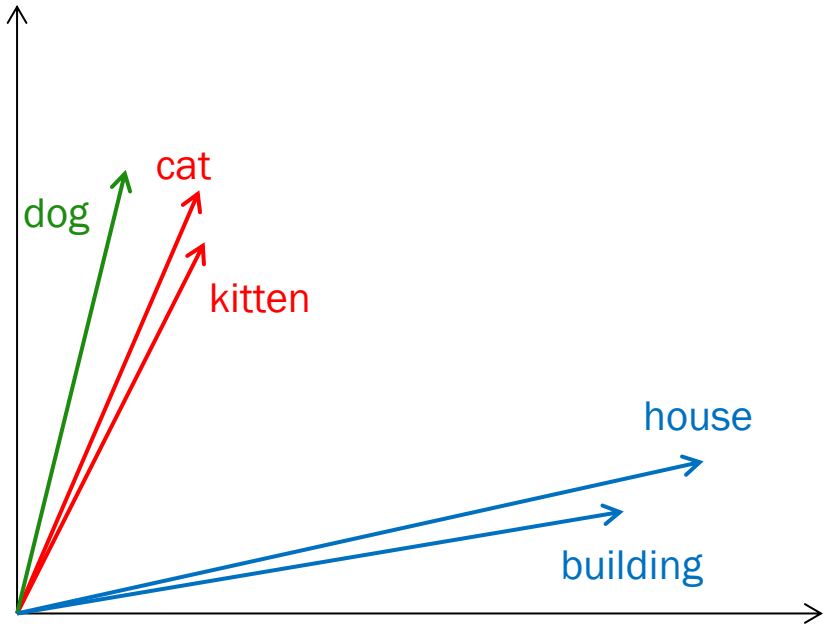
- Words with similar meaning will have similar values, e.g.

```
similarity(cat, kitten) = 0.91
similarity(cat, house)  = 0.18
```

- Doesn't differentiate between different meanings of the same word, so V() = V().

Words #2: Word embeddings (Word2Vec)

- If we plot these vectors in a multidimensional space, we'll see that similar words are closer together.



- Vectors capture relationships between words:

```
kitten – cat + dog = puppy
king – man + woman = queen
```

- Words with similar meaning will have similar values, e.g.

```
similarity(cat, kitten) = 0.91
similarity(cat, house)  = 0.18
```

- Doesn't differentiate between different meanings of the same word, so V() = V().

Document #2: Mean Pooling

The semantic content of a document is defined as the average of its word vectors:

```
cat     = [ 0.92,  0.99, 0.59, -0.91, …, 0.32]
dog     = [ 0.92,  0.21, 0.62, -0.92, …, 0.44]
house   = [-0.45, -0.72, 0.66,  0.98, …, 0.84]
-----------------------------------------------
```

V_{doc} = [0.46, 0.16, 0.62, -0.28, …, 0.53]

- Also ignores word order, so `V("the dog chased the cat") = V("the cat chased the dog").`

Words #3: Contextual Embeddings (e.g. BERT)

Vectors are dynamic, so their values change based on context:

*The **bat** was sleeping in the cave.* `bat = [0.45, -0.39, -0.01, …, 0.10]`

*The boy hit the ball with a **bat**.* `bat = [0.08, -0.12,  0.02, …, 0.82]`

- Differentiates between different meanings of the same word, so `V(`  `) ≠ V(`  `).`
- Can generate vectors for unseen words by splitting the word into smaller units (tokens), e.g.

```
V("batmobile")
= V("bat") + V("mobile")
= [0.81, 0.98, 0.24, …, -0.95]
```

Document #3: Mean Pooling or Internal Representation

We can average word vectors or use an internal custom representation of the whole text, e.g.

`[-0.87, 0.04, -0.52, …, 0.61]` or V_{doc} = **average(**`V(The), V(boy), V(hit), V(the), V(ball), V(with), V(a), V(bat)`**)**

- Preserves word order, so `V("the dog chased the cat")` ≠ `V("the cat chased the dog")`.

Building machine learning models for text

Sentence embeddings are numerical representations of text, so we can use them as input data to train a model, such as a classifier. This is like using features.

Example: Train a classifier to detect spam from email subjects.

Email subject	Embedding	Label
Draft agenda for next week's research seminar	`[-0.15, 0.42, 0.08, ..., -0.21]`	Not Spam
URGENT: You have won a £1,000 gift card!	`[0.82, -0.14, 0.05, ..., 0.65]`	Spam
Project update meeting scheduled for Monday	`[-0.31, 0.55, 0.12, ..., -0.12]`	Not Spam
Can you please review the attached document?	`[-0.25, 0.48, -0.11, ..., -0.30]`	Not Spam
Flight confirmation for your upcoming journey	`[-0.18, 0.39, 0.07, ..., 0.11]`	Not Spam
Final notice: Verify your bank details now	`[0.68, -0.09, 0.29, ..., 0.77]`	Spam
Weekly digest: Department news and announcements	`[-0.05, 0.22, -0.19, ..., -0.15]`	Not Spam
Quick question regarding the Python script	`[-0.38, 0.51, -0.02, ..., -0.18]`	Not Spam
Earn £5,000 a week working from home!	`[0.89, -0.33, 0.14, ..., 0.42]`	Spam
Notes from yesterday's team sync	`[-0.42, 0.61, -0.03, ..., -0.05]`	Not Spam

Let’s try it!

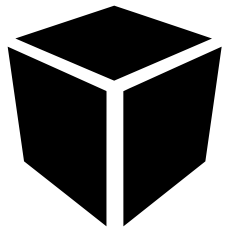
s.id/css26

Language models

A language model is a probability distribution over sequences of words.

Language models can assign probabilities to sequences of words and generate new sequences based on probabilities.

Language models



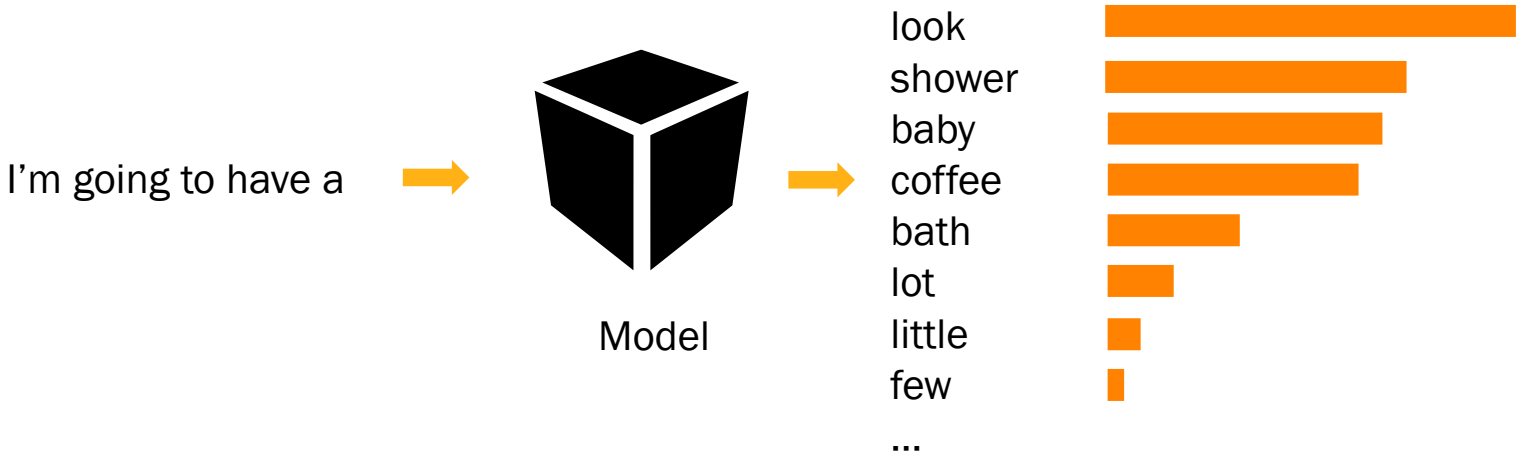
Model

Language models

I’m going to have a 

Model

Language models



Language models

N-grams

Context window of 2, 3, 4 or 5 words

Predict only from left to right

Language models

N-grams

Context window of 2, 3, 4 or 5 words

Predict only from left to right

She bought a car but she doesn't know how to…

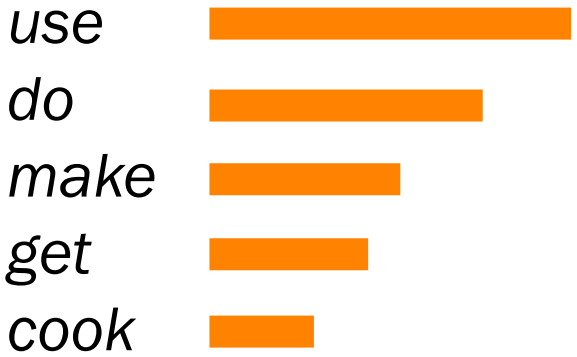
Language models

N-grams

Context window of 2, 3, 4 or 5 words

Predict only from left to right

She bought a car but she doesn't know how to



Language models

Neural models

Take all previous words into account

Look at left and right context

Can predict words in any position

She bought a car but she doesn't know how to…

Language models

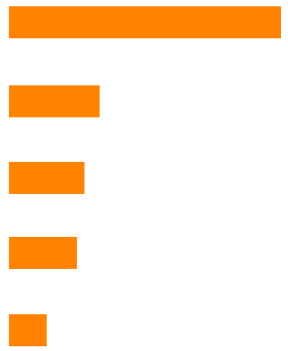
Neural models

Take all previous words into account

Look at left and right context

Can predict words in any position

She bought a car but she doesn't know how to



Language models

Probability of sequences

Language models

Probability of sequences

He is *always* late.

He *always* is late.

Language models

Probability of sequences

He is *always* late. 

He *always* is late. 

Language models

Probability of sequences

He is *always* late. 

He *always* is late. 

In his *nineties*, he was still active.

In his *nine teeth*, he was still active.

In his *nine tease*, he was still active.

Language models

Probability of sequences

He is *always* late.

He *always* is late.

In his *nineties*, he was still active.

In his *nine teeth*, he was still active.

In his *nine tease*, he was still active.

Large Language Models (LLMs)

- Large Language Models generate text one word at a time by predicting what comes next based on context.
- They are trained on vast amounts of online text from many different languages.
- They are refined with human feedback so they can follow instructions and answer questions accurately.
- They can handle a wide variety of tasks, from writing and summarising to coding and reasoning.
- Popular examples include ChatGPT, Gemini, Claude, and DeepSeek.





Let’s try it!

s.id/css26

Group assignments

Instructions

1. Each team must choose one problem from the provided list (each team must work on a different problem).
2. Solve the problem using machine learning and the given dataset (using scikit-learn or other relevant libraries).
3. Try at least 2 different machine learning algorithms and compare their performance.
4. Prepare a slide deck and deliver a 15-minute presentation during the next session.

Challenges (choose one)

1. **Poisonous mushrooms**
   Determine if a mushroom is poisonous or not.
2. **Personality prediction**
   Predict a person's personality type.
3. **Twitter sentiment analysis**
   Determine the sentiment of Twitter posts.
4. **Loan approval**
   Determine if a client's loan application will be approved.
5. **Star types**
   Classify stars.
6. **Crop recommendation**
   Recommend the most suitable crop for a farm.

See you on Wednesday!

Mariano.Felice@cl.cam.ac.uk